

프로젝트 결과 보고서

기본 정보

항목	내용
팀명	취업시켜조
작품명	단번에
팀원	권혁태 · 김동현 · 이수진
개발 기간	3개월
GitHub	https://github.com/Dan-burn-go
서비스 URL	https://goseoul.today
대표 연락처	010-8551-9539

팀원 역할 분담

이름	역할
김동현	Leader · PO · SM
이수진	Frontend · Backend · DevOps
권혁태	Backend · AI · DevOps

프로젝트 개요

단번에는 서울시 주요 관광지 122곳의 실시간 혼잡도를 수집·분석하고, AI 기반 혼잡 원인 분석과 함께 대체 장소 및 대중교통 경로를 추천하는 서울 관광 혼잡도 분석 플랫폼입니다.

공공데이터 API를 통해 5분마다 혼잡도 데이터를 수집하고, 혼잡 상태 전이 시 AI가 원인을 분석하여 사용자에게 반경 2km 내 대체 장소와 이동 경로를 함께 제공합니다.

개발 일정

프로젝트는 4개의 스프린트로 진행되었으며, GitHub Projects 칸반 보드로 태스크를 관리했습니다.

스프린트	단계	달성도	주요 완료 항목
S1	설계 · 기획	9 / 9	Figma 디자인, DB 스키마, 공공데이터 API POC, 와이어프레임, API 설계, 기술 스택 확정, 개발환경 세팅

스프린트	단계	달성도	주요 완료 항목
S2	핵심	26	CI/CD 파이프라인, Observability(Grafana+OTel), Docker Compose, API Gateway, 혼잡도 서비스(서울시 API), RabbitMQ 도입, DLQ Worker, OpenAI 클라이언트, service-congestion↔AI 메시지 연동, 카카오 맵 마커 렌더링
	기능	/	
	구현	26	
S3	기능	13	랭킹 페이지, MCP 서버(DuckDuckGo), Function Calling 통합 및 Fallback 정책, 대체지역 실데이터 연동, API Composition 패턴, 상세 페이지 구현
	고도	/	
	화	13	
S4	마무	8 /	주변 장소 조회(캐싱), 대체지 추천 뷰, ODsay 경로 API, 문화 행사 뷰, 익명 구독 + Web Push 알림, 주변 장소 테스트 코드
	리·	11	
	운영		

S4 진행 중 항목: 모바일 반응형 개선 · 혼잡도 통계 그래프 · 상세 페이지 AI 리포트 연동

1. 기술 스택

Backend

분류	기술
언어 / 런타임	Java 21
프레임워크	Spring Boot 3.4.5, Spring Cloud 2024.0.2
AI 서비스	Python 3.12, FastAPI
데이터베이스	MySQL 8.0, Redis 7
메시지 브로커	RabbitMQ
인프라	Docker Compose, GitHub Actions (CI/CD)
모니터링	Prometheus, Grafana
보안	Infisical (시크릿 관리)

Frontend

분류	기술
언어	TypeScript 5.9
프레임워크	React 19
빌드 도구	Vite 7
스타일링	Tailwind CSS 4
지도	Kakao Maps API

분류	기술
HTTP 통신	Axios

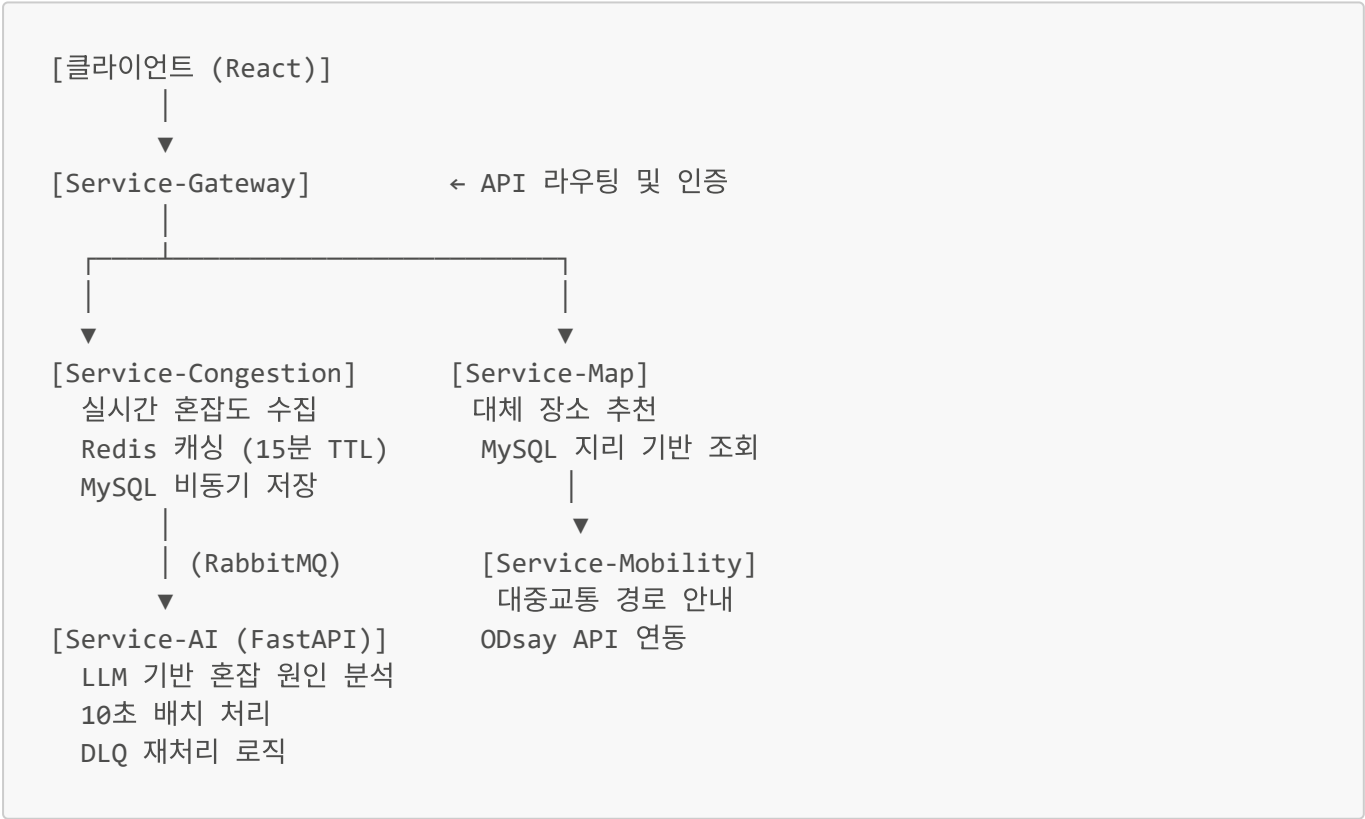
운영 환경

분류	도구	역할
모니터링	Prometheus + Grafana	서비스 메트릭 수집·시각화, 디스크·메모리 임계치 알림
분산 추적	OpenTelemetry (OTel)	마이크로서비스 간 요청 흐름 추적
컨테이너 관리	Portainer	Docker 컨테이너 상태 확인, 로그 열람, 장애 원인 파악
배포	GitHub Actions	PR 머지 시 자동 빌드·배포 (CI/CD)
시크릿 관리	Infisical	API 키 등 민감 정보 중앙 관리

2. 시스템 아키텍처

전체 구조

단번에는 **마이크로서비스 아키텍처(MSA)** 와 **이벤트 기반 아키텍처(EDA)** 를 결합한 구조로 설계되었습니다. RabbitMQ를 메시지 브로커로 활용하여 Java 서비스와 Python AI 서비스를 분리함으로써 서비스 간 의존성을 제거하고, 한 서비스의 장애가 전체 시스템으로 전파되는 것을 차단합니다.



서비스 구성

서비스	역할
Service-Gateway	API 진입점, 라우팅
Service-Congestion	서울시 공공 API 수집, Redis 캐싱, MySQL 저장
Service-AI	Python/FastAPI, LLM 혼잡 원인 분석
Service-Map	반경 2km 내 대체 장소 추천
Service-Mobility	ODsay API 기반 대중교통 경로 제공
Service-Common	공통 유틸리티

데이터 파이프라인

1. **수집**: 서울시 공공 API에서 5분 주기로 122개 관광지 혼잡도 수집
2. **캐싱**: Redis에 15분 TTL로 최신 데이터 유지
3. **저장**: MySQL에 비동기 영속화, 매일 03:00 오래된 데이터 자동 정리
4. **분석**: 혼잡도가 **BUSY 상태로 전이**될 때만 RabbitMQ 이벤트 발행 → AI 서비스가 10초 배치 윈도우로 묶어 처리하여 API 비용 절감

3. 기능 소개

3.1 실시간 혼잡도 지도

서울시 122개 관광지의 실시간 혼잡도를 카카오 맵 위에 색상 마커로 시각화하는 핵심 화면입니다.

백엔드 — 데이터 수집 및 제공

`CongestionService`는 Redis와 MySQL 이중 레이어로 데이터를 관리합니다.

- `saveAllToRedis()` / `saveAllToDb()`: 공공 API 수집 데이터를 Redis에 즉시 캐싱하고 MySQL에 비동기 저장
- `findAll()`: **Single-Flight 패턴** 적용 — Redis 미스 발생 시 동시 요청들을 직렬화하여 첫 번째 요청만 DB에 접근하고 나머지는 락 해제 후 Redis를 재조회, 커넥션 풀 고갈 방지
- `findByAreaCode()`: 개별 장소 조회 시 Redis 우선 조회 → 미스 시 DB Fallback

제공 API:

- `GET /api/congestion` — 122개 장소 전체 혼잡도 일괄 반환
- `GET /api/congestion/{areaCode}` — 특정 장소 혼잡도 (예: `POI001`)

프론트엔드 — 지도 시각화

`useCongestionMarkers.ts` 커스텀 훅이 API 응답을 카카오 맵 마커로 변환합니다. `HomePage.tsx`는 지도·필터·줌·범례·랭킹 카드를 하나의 `h-dvh` 전체화면 레이아웃으로 통합 렌더링합니다.

혼잡도 단계	색상
여유	초록

혼잡도 단계	색상
보통	파랑
약간 붐빔	주황
붐빔	빨강

- `useHotspotsFilter.ts`: 카테고리 필터 상태 관리 — 원하는 유형의 장소만 선택적 탐색
- `useMapControls.ts`: 줌 레벨 제어 및 줌에 따른 마커 크기 동적 조정
- `useRanking.ts`: 실시간 혼잡 TOP N / 여유 TOP N 랭킹 카드 데이터 관리
- 모바일 환경에서는 범례를 약어로, 데스크탑에서는 전체 명칭으로 표시하는 반응형 처리

3.2 AI 혼잡 원인 분석

혼잡도 변화를 감지하고 LLM이 원인을 분석하는 기능입니다. 불필요한 AI API 호출을 최소화하는 구조 설계에 초점을 맞췄습니다.

엣지 감지 (Edge Detection)

`CongestionStateTracker`가 혼잡도 상태 전이를 감지합니다.

- `filterAreaCodesForAnalysis()`: 수집된 데이터 중 BUSY 상태인 장소를 추출 → 이전 상태를 **Bulk 조회** (N+1 방지)하여 non-BUSY → BUSY 전이가 일어난 장소 코드만 선별
- `filterAreaCodesForRelaxedNotification()`: 반대로 BUSY → non-BUSY 전이 감지 (혼잡 해소 알림용)
- `getPreviousLevels()`: Redis `multiGet`으로 단일 네트워크 호출로 이전 상태 일괄 조회 → 캐시 미스 항목만 DB 보완 조회

AI 분석 및 캐싱

- OpenAI Function Calling + MCP 연동으로 AI가 외부 데이터 도구를 직접 호출하여 원인 분석
- 전이 감지 이벤트를 RabbitMQ로 발행, AI 서비스가 **10초 배치 윈도우**로 묶어 처리 → API 호출 횟수 절감
- `AIService.getLatestAiReport(areaCode)`: 분석 결과를 `ai-report:{areaCode}` 키로 Redis 캐싱 → DB Fallback 구조로 빠른 응답 보장
- `AnomalyDetector`: 이상 혼잡도 감지 로직 별도 분리

혼잡도 통계 분석 API

`CongestionAnalysisService`가 누적 데이터를 통계화하며, 4개 엔드포인트를 제공합니다.

엔드포인트	설명
<code>GET /api/congestion/analysis/hourly/{areaCode}?days=7</code>	시간대별(0~23시) 혼잡도 추이
<code>GET /api/congestion/analysis/daily/{areaCode}?days=7</code>	요일별 혼잡도 패턴
<code>GET /api/congestion/analysis/ranking/busiest?limit=10</code>	현재 가장 붐비는 장소 TOP N
<code>GET /api/congestion/analysis/ranking/relaxed?limit=10</code>	현재 가장 여유로운 장소 TOP N

`HourlyAvgCacheService`가 시간별 평균값을 별도 캐싱하여 통계 쿼리 성능을 보완합니다.

3.3 대체 장소 추천

현재 혼잡한 장소 대신 방문할 수 있는 유사 장소를 실시간 혼잡도 기준으로 추천합니다.

추천 로직 (**AlternativeLocationService**)

`getAlternativeLocations(areaCode)` 메서드는 다음 4단계로 동작합니다.

1. **코드 변환**: `LocationCodeMapper`로 `areaCode` → `locationId` 변환, 미매핑 시 404 예외
2. **DB 조회**: `AlternativeLocationJpaRepository`에서 우선순위 순으로 대체 장소 목록 조회
3. **병렬 혼잡도 조회**: Project Reactor의 `Flux.flatMap`으로 각 대체 장소의 실시간 혼잡도를 **병렬 비동기** 호출 (10초 타임아웃, 실패 시 null Fallback)
4. **정렬 및 반환**: 혼잡도 수준 기준 오름차순 정렬 후 반환, 혼잡도 null인 장소는 최하위 배치

혼잡도 우선순위 기준:

순위	혼잡도
1	여유
2	보통
3	약간 붐빔
4	붐빔

프론트엔드 처리 (**HomePage.tsx**)

- `alternativeState`로 추천 장소 목록 상태 관리
- `AbortController`를 활용해 이전 요청을 취소하고 최신 요청만 유지하는 중복 요청 방지 처리
- 추천 배너를 닫거나 다른 장소를 검색하면 상태 초기화
- 대체 장소 마커 클릭 시 상세 페이지로 이동 — 이 때 navigation state를 제거하고 상세 페이지가 직접 API를 호출하도록 설계하여 빈 데이터 표시 버그 방지

주변 문화 행사 정보는 `CultureEventService`와 `useCultureEvents.ts` 혹은 통해 좌표 기반으로 연동됩니다.

3.4 대중교통 경로 안내

대체 장소 또는 목적지까지의 대중교통 이동 경로를 제공합니다.

API 선정 배경

자동차·도보·대중교통 경로 중 **대중교통만 제공**하기로 결정했습니다. 서비스 대상이 서울 관광지를 이동하는 사용자임을 고려할 때 대중교통이 가장 실용적이며, 무료로 제공되고 일일 호출 제한이 넉넉한 **ODsay API**를 선정했습니다.

서비스 구조 (**OdsayService**)

```
OdsayApiRequest
↓
```

```

OdsayApiClient      ← ODSay 외부 API 호출
    ↓
OdsayResponseMapper ← 응답 필드 매핑 및 가공
    ↓
TransitRouteResponse ← 클라이언트에 반환

```

- **OdsayResponseMapper**: ODSay API 응답은 다층 구조로 이루어져 있어, 클라이언트에 필요한 필드만 추출하여 단순화된 DTO로 변환
- 모든 응답 DTO에 `@JsonIgnoreProperties(ignoreUnknown = true)` 명시 적용 — 전역 설정 변경에도 안전하게 동작하도록 코드 의도를 명시적으로 표현
- **DetailPage.tsx**의 상세 페이지에서 경로 정보를 렌더링하며, 데이터 로딩 중에는 스켈레톤 UI로 처리

3.5 주변 장소 조회

상세 페이지에서 현재 위치 기반으로 반경 1km 내 주변 장소를 카테고리별로 조회하는 기능입니다.

API (**NearbyPlaceController**)

```
GET /api/map/nearby-places?areaCode=POI009&categoryCode=ALL
```

categoryCode는 **ALL** 또는 아래 4가지 Kakao 카테고리 코드를 허용하며, 미입력 시 기본값 **ALL**이 적용됩니다.

코드	카테고리
FD6	음식점
CE7	카페
AT4	관광명소
CT1	문화시설

`@Validated` + `@Pattern`으로 허용되지 않는 카테고리 코드 요청을 컨트롤러 진입 전에 차단합니다.

서비스 로직 (**NearbyPlaceService**)

- **단일 카테고리**: `fetchWithCache(areaCode, categoryCode)` 호출 → Redis 캐시 확인 → 미스 시 Kakao API 호출
- **ALL 카테고리**: 4개 카테고리 코드를 `parallelStream()`으로 병렬 조회 후 `flatMap`으로 결과 합산
- **캐싱 전략**: 결과를 1시간 TTL로 Redis에 저장. **빈 배열은 캐싱하지 않아** 다음 요청 시 Kakao API 재호출 보장
- Kakao API 장애 시 예외를 던지지 않고 빈 리스트를 반환하는 Graceful Degradation 적용
- **KakaoLocalApiClient**: 검색 반경 1,000m, 최대 15개 결과, 연결/읽기 타임아웃 5초

3.6 주변 문화 행사 조회

장소 상세 페이지에서 해당 좌표 기준 반경 1km 내 진행 중인 문화 행사 목록을 제공하는 기능입니다.

백엔드 (CultureEventService)

- `getCultureEvents(Double latitude, Double longitude)`: 좌표를 받아 `findEventsWithinRadius(lat, lon, 1000.0)`로 DB에서 반경 내 행사 조회
- 행사 정보: 제목, 장소, 분류, 날짜, 요금, 링크, 이미지, 좌표 10개 필드 반환
- `@Transactional(readOnly = true)` 적용

프론트엔드 (useCultureEvents.ts)

- 위도·경도를 의존성으로 받아 좌표가 확정된 시점에만 API 호출 (null 좌표 요청 방지)
- `AbortController`로 컴포넌트 언마운트 또는 좌표 변경 시 진행 중인 요청 취소
- `loading / success / error` 3가지 상태로 관리하여 `DetailPage`에서 로딩·에러·정상을 각각 다른 UI로 렌더링
- 정상 응답 시 반응형 그리드(모바일 2열, 데스크탑 3열)로 행사 카드 표시

3.7 혼잡도 랭킹

전체 122개 관광지를 혼잡도 기준으로 정렬하여 순위를 제공하는 전용 페이지입니다.

프론트엔드 (RankingPage.tsx + useRanking.ts)

- **탭 전환**: "혼잡도 높은 순" / "혼잡도 낮은 순" 탭을 URL 파라미터(`?tab=busiest`)로 관리하여 새로고침 후에도 선택한 탭이 유지됨
- **useRanking** **혹**: `useReducer`로 `reset / success / error` 3가지 액션 패턴 적용. 탭 변경 시 `AbortController`로 이전 요청을 즉시 취소하고 새 요청 시작
- **랭킹 카드**: 1~3위에 금·은·동 배지 표시, 장소명·카테고리 해시태그·평균 인구수·색상 혼잡도 레이블 포함
- 데이터 로딩 중 5개 스켈레톤 플레이스홀더 표시

백엔드 연동

- `GET /api/congestion/analysis/ranking/busiest?limit=122`
- `GET /api/congestion/analysis/ranking/relaxed?limit=122`
- `CongestionAnalysisService.buildRanking()`: atomic counter로 순위 번호 부여, `getLevelScore()`로 혼잡도 enum을 수치로 변환하여 정렬

3.8 핫스팟 탐색 (장소 목록)

서울 122개 관광지를 카드 형태로 탐색하는 목록 페이지입니다.

프론트엔드 (HotspotsPage.tsx + useHotspotsFilter.ts)

검색

- 실시간 텍스트 검색으로 장소명 필터링
- 한글 입력 시 조합 중인 글자가 즉시 검색에 반영되는 문제를 방지하기 위해 **IME Composition Event** 핸들링 적용 (`onCompositionStart / onCompositionEnd`)
- 검색어와 카테고리 필터를 URL 파라미터로 관리하여 뒤로가기 후에도 상태 유지

카테고리 필터

카테고리	아이콘
전체	Sparkles
카페	Coffee
쇼핑	ShoppingBag
공원	Trees
문화	Landmark

페이지네이션

- 대량 목록을 페이지 단위로 분할 표시
- 페이지 수가 많을 경우 말줄임(...) 처리로 UI 오버플로우 방지
- 이전/다음 화살표 네비게이션 및 현재 페이지 하이라이트

장소 카드

- 반응형 그리드: 모바일 1열 → 태블릿 2열 → 데스크탑 3열
- 커버 이미지 로드 실패 시 Fallback 처리
- 카테고리 배지 오버레이 및 실시간 상태 펄스 애니메이션
- 카드 클릭 시 해당 장소 상세 페이지로 이동

3.9 실시간 데이터 수집 파이프라인

서울시 공공 API에서 혼잡도 데이터를 주기적으로 수집하고 저장·분석하는 자동화 파이프라인입니다.

수집 스케줄러 (CongestionScheduler)

`fetchAndSave()` 메서드는 `fixedRateString`으로 외부 설정값(`congestion.scheduler.interval`)을 받아 실행 주기를 코드 수정 없이 조정할 수 있도록 설계되어 있습니다. 초기 딜레이는 기본 10초(`initial-delay:10000`)로 설정되어 서비스 구동 직후의 불안정 구간을 회피합니다.

실행 시 수행하는 작업 순서:

```

서울시 공공 API 호출
    ↓
응답 → DTO 변환
    ↓
AnomalyDetector.detectAnomalies() - 이상 혼잡 감지
    ↓
Redis 저장 (최신 데이터 캐싱)
    ↓
MySQL 비동기 저장 (이력 영속화)
    
```

이상 혼잡 감지 (AnomalyDetector)

단순 BUSY 여부가 아닌, **현재 인구 수 / 시간대별 평균 인구 수의 비율**이 설정 임계치(기본 1.5배)를 초과할 때만 이상 혼잡으로 판정합니다. 임계치는 `congestion.anomaly.max-people-ratio` 설정값으로 조정 가능합니다.

중복 이벤트 방지 — Armed Flag 패턴

동일 장소에서 혼잡이 지속되는 동안 이벤트가 반복 발행되는 문제를 Redis SETNX 기반 Armed Flag로 해결합니다.

- `tryArm(areaCode)`: Redis SETNX 원자적 연산으로 플래그 등록 — 최초 1회만 성공, 이미 등록된 경우 실패 반환 → 중복 이벤트 발행 차단
- Armed Flag TTL: 24시간
- `releaseArmedForNonBusy()`: 혼잡 해소 시 커서 기반 Redis 스캔으로 해당 장소의 플래그 삭제 → 혼잡 재발 시 재발행 가능

기준 데이터 부재 처리: `HourlyAvgCacheService`에서 시간대 평균값을 조회할 수 없을 경우 보수적으로 이상 상황으로 간주하여 이벤트 발행

데이터 자동 정리 (`DataCleanupScheduler`)

- 매일 03:00(기본값) 실행, 보존 기간(기본 7일) 이전 혼잡도 데이터 자동 삭제
- `congestion.cleanup.retention-days`, `congestion.cleanup.cron` 설정값으로 운영 환경별 커스터마이징 가능

3.10 RabbitMQ 이벤트 기반 AI 분석 파이프라인

Java 서비스와 Python AI 서비스를 RabbitMQ로 완전히 분리하는 이벤트 파이프라인입니다. 두 서비스는 서로 직접 호출하지 않으며, 메시지 브로커를 통해서만 통신합니다.

이벤트 발행 (`CongestionEventPublisher`)

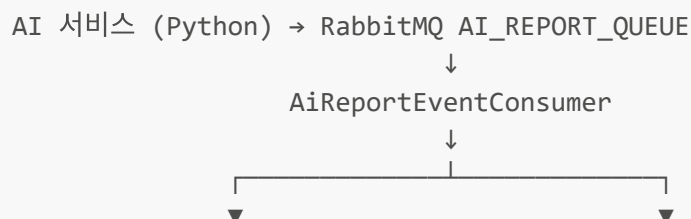
`RabbitTemplate.convertAndSend(exchange, routingKey, payload)` 방식으로 두 종류의 이벤트를 발행합니다.

이벤트	발행 조건	포함 데이터
<code>CongestionBusyEvent</code>	non-BUSY → BUSY 상태 전이 감지	areaCode, 발생 시각
<code>CongestionAnomalyEvent</code>	현재/평균 인구 비율 > 임계치	areaCode, 현재 인구, 평균 인구, 비율

Exchange 이름·라우팅 키는 `RabbitMqConfig`에 상수로 중앙 관리하여 하드코딩 방지

AI 분석 결과 수신 (`AiReportEventConsumer`)

AI 서비스가 분석을 마치면 결과를 RabbitMQ를 통해 역방향으로 발행하고, `@RabbitListener`가 이를 소비합니다.



```
aiReportRepository                Redis 캐싱
.insertIfAbsent()                 key: "ai-report:{areaCode}"
(중복 메시지 무시)              TTL: 4시간
```

역등성 보장: `insertIfAbsent()`가 0을 반환하면 중복 메시지로 판단하고 무시. DB 저장과 Redis 캐싱을 독립적으로 처리하여 캐싱 실패가 DB 저장에 영향을 주지 않음

3.11 문화 행사 자동 동기화

서울시 문화 행사 정보를 주기적으로 수집하여 DB에 최신 상태를 유지하는 배치 처리 시스템입니다.

동기화 스케줄러 (`EventSyncScheduler`)

`@Scheduled(cron = "0 0 * * * *")`로 매시간 실행하며, 운영 환경에서는 새벽 3시로 변경 예정입니다.

배치 수집 로직 (`EventService.fetchAndSyncEvents()`)

1. 기존 행사 전체 로드 → 복합키(제목|장소|시작일) Map으로 인덱싱
2. 서울 문화 행사 API 1,000건 단위 페이지 요청
3. 첫 응답의 `listTotalCount`로 `maxBatches` 동적 계산
→ `maxBatches = ceil(listTotalCount / 1000)`
→ 불필요한 API 호출 사전 차단
4. `EventUpsertService`에 배치 위임 (`upsert` 처리)
5. 반환 건수 < `batchSize` 조건 충족 시 조기 종료
6. 종료일이 오늘 이전인 만료 행사 자동 삭제

중복 방지 업서트 (`EventUpsertService.upsertEventBatch()`)

- 복합키(제목 + 장소 + 시작일)로 기존 행사 여부 확인
- 이미 존재하면 변경된 필드만 업데이트, 없으면 신규 삽입
- 날짜 파싱 실패/과거 행사는 조용히 건너뛸(Graceful Skip)
- 좌표 데이터에 ~ 구분자가 포함된 비표준 포맷을 `parseCoordinate()`로 처리
- 트랜잭션 단위를 배치별로 분리하여 일부 실패 시 해당 배치만 롤백

4. API 명세 요약

Service-Congestion

메서드	엔드포인트	설명	주요 파라미터
GET	<code>/api/congestion</code>	전체 122개 장소 혼잡도 조회	-
GET	<code>/api/congestion/{areaCode}</code>	장소별 혼잡도 조회	<code>areaCode</code> (예: POI001)

메서드	엔드포인트	설명	주요 파라미터
GET	/api/congestion/{areaCode}/ai-report	AI 혼잡 원인 분석 리포트	areaCode
GET	/api/congestion/analysis/hourly/{areaCode}	시간대별(0~23시) 혼잡도 추이	areaCode, days (기본 7)
GET	/api/congestion/analysis/daily/{areaCode}	요일별 혼잡도 패턴	areaCode, days (기본 7)
GET	/api/congestion/analysis/ranking/busiest	현재 혼잡도 높은 장소 TOP N	limit (기본 10)
GET	/api/congestion/analysis/ranking/relaxed	현재 혼잡도 낮은 장소 TOP N	limit (기본 10)

Service-Map

메서드	엔드포인트	설명	주요 파라미터
GET	/api/map/alternative-location	대체 장소 추천 3개 조회	areaCode
GET	/api/map/culture-events	반경 1km 내 문화 행사 조회	latitude, longitude
GET	/api/map/nearby-places	반경 1km 내 주변 장소 조회	areaCode, categoryCode (기본 ALL)

categoryCode 허용값: ALL · FD6(음식점) · CE7(카페) · AT4(관광명소) · CT1(문화시설)

Service-Mobility

메서드	엔드포인트	설명	주요 파라미터
GET	/api/mobility/route	대중교통 경로 조회	originLat, originLng, destLat, destLng

5. ERD

service-mobility · service-ai · service-gateway는 DB 없이 외부 API를 직접 호출하는 stateless 서비스입니다.

service-congestion

congestion

키	한글	컬럼명	타입	NOT NULL
PK	식별자	id	BIGINT	✓

키	한글	컬럼명	타입	NOT NULL
-	지역 코드	area_code	VARCHAR(20)	✓
-	지역명	area_name	VARCHAR(100)	✓
-	혼잡도 단계	congestion_level	VARCHAR(20)	✓
-	혼잡도 메시지	message	TEXT	-
-	실시간 인구 최솟값	live_pop_min	INT	-
-	실시간 인구 최댓값	live_pop_max	INT	-
-	마지막 갱신 시각	updated_at	DATETIME	✓

ai_report

키	한글	컬럼명	타입	NOT NULL
PK	식별자	id	BIGINT	✓
FK	지역 코드 (→ congestion.area_code)	area_code	VARCHAR(20)	✓
-	분석 내용	content	TEXT	✓
-	생성 시각	created_at	DATETIME	✓

notification_subscriptions

키	한글	컬럼명	타입	NOT NULL
PK	식별자	id	BIGINT	✓
-	지역 코드	area_code	VARCHAR(20)	✓
-	FCM 토큰	fcm_token	VARCHAR(255)	✓
-	구독 생성 시각	created_at	DATETIME	✓

service-map

locations

키	한글	컬럼명	타입	NOT NULL
PK	식별자	id	BIGINT	✓
-	지역 코드	area_code	VARCHAR(20)	✓
-	장소명	name	VARCHAR(100)	✓
-	위도	latitude	DOUBLE	✓
-	경도	longitude	DOUBLE	✓
-	주소	address	VARCHAR(255)	-

키	한글	컬럼명	타입	NOT NULL
-	카테고리	category	VARCHAR(50)	-

alternative_locations

키	한글	컬럼명	타입	NOT NULL
PK	식별자	id	BIGINT	✓
FK	원본 지역 코드 (→ locations.area_code)	area_code	VARCHAR(20)	✓
-	대체 지역 코드	alt_area_code	VARCHAR(20)	✓
-	거리 (m)	distance	DOUBLE	-
-	등록 시각	created_at	DATETIME	✓

events

키	한글	컬럼명	타입	NOT NULL
PK	식별자	id	BIGINT	✓
-	행사명	title	VARCHAR(200)	✓
-	카테고리	category	VARCHAR(50)	-
-	시작일	start_date	DATE	-
-	종료일	end_date	DATE	-
-	장소명	location_name	VARCHAR(100)	-
-	위도	latitude	DOUBLE	-
-	경도	longitude	DOUBLE	-
-	상세 URL	url	VARCHAR(500)	-
-	등록 시각	created_at	DATETIME	✓

stores

키	한글	컬럼명	타입	NOT NULL
PK	식별자	id	BIGINT	✓
UK	카카오 장소 ID	place_id	VARCHAR(50)	✓
-	상호명	name	VARCHAR(100)	✓
-	카테고리	category	VARCHAR(50)	-
-	위도	latitude	DOUBLE	✓
-	경도	longitude	DOUBLE	✓

키	한글	컬럼명	타입	NOT NULL
-	주소	address	VARCHAR(255)	-
-	등록 시각	created_at	DATETIME	✓

6. 트러블슈팅

6.1 RabbitMQ DLQ로 메시지 무손실 달성

문제: 외부 AI API의 호출량 제한(Rate Limiting)으로 인해 메시지 유실 발생 위험

해결: Dead Letter Queue(DLQ) 구현

- 정상 처리 경로와 실패 경로를 분리
- DLQ 재처리 시 헤더로 재시도 횟수를 추적하여 무한 루프 차단
- 토큰 버킷 사용률 기반으로 소프트 한도(Soft Limit) 판정

결과: API Rate Limiting 환경에서도 메시지 무손실 달성

6.2 서버 디스크 포화로 인한 CD 파이프라인 장애

문제: 로그 정리 정책 없이 로그를 무한히 축적하다 디스크 포화 → 신규 컨테이너 기동 불가 → GitHub Actions CD 파이프라인 전체 중단

해결:

- Portainer와 Grafana 모니터링 툴로 원인 파악
- 오래된 로그 수동 삭제 후 서비스 복구
- 디스크 사용량이 일정 임계치를 넘으면 오래된 로그를 자동 삭제하는 정책 수립
- 인프라 메트릭에 Grafana 알림 임계치 설정

교훈: 로그 보존·회수 정책은 기능 개발 단계에서 함께 설계해야 함

6.3 Git 브랜치 히스토리 충돌

문제: 배포 후 롤백 과정에서 GitHub Revert 버튼 남용 → dev·main 브랜치 싱크 깨짐 → 코드 충돌 빈번 발생

원인 분석: GitHub의 Revert 버튼은 단순 롤백이 아닌 PR 자동 생성 및 Merge를 수행하는 특수 기능으로, Protected 브랜치에서도 동작함. Revert of Revert 이후 동일 파일 수정 시 충돌 발생

해결:

- main·dev 브랜치에 Branch Protection Rule 적용 (최소 1인 승인 후 머지)
- Protected 브랜치 환경에서 git merge origin/main으로 충돌을 새 브랜치에서 먼저 해소 후 PR하는 워크플로우 정착

6.4 Redis 장애 시 서비스 전체 500 에러

문제: @Cacheable 어노테이션 사용 중 Redis 장애 발생 시 API 전체가 500 에러 반환

해결: StringRedisTemplate을 활용한 수동 캐싱으로 전환 + try-catch 기반 Fallback 로직 구현

- Redis 장애 시 캐시를 건너뛰고 DB에서 직접 조회
- Look Aside / Write Back / Write Through 캐싱 전략 검토 후 서비스 특성에 맞게 적용

6.5 React StrictMode로 인한 스켈레톤 UI 버그

문제: 홈 화면 오버레이 카드에서 데이터가 없는데 스켈레톤이 사라지는 버그

원인: React StrictMode가 개발 환경에서 컴포넌트를 mount → unmount → mount 반복 시 unmount 시점에 API 요청이 abort되었음에도 finally 블록이 실행되어 setLoading(false) 호출

해결: abort 여부를 확인하여 상태 업데이트를 차단하는 패턴 적용

교훈: 비동기 요청 설계 시 React Effect 생명주기를 함께 고려해야 함. Network 탭 단독이 아닌 React DevTools를 병행하여 상태 변화 흐름을 교차 추적하는 것이 디버깅의 핵심

6.6 Kakao Maps SDK가 컴포넌트를 덮는 렌더링 문제

문제: 지도 위에 표시해야 할 컴포넌트가 화면에 나타나지 않음

원인: 렌더링 자체는 되고 있었으나, Kakao Maps SDK 내부 요소가 컴포넌트 위를 z-index로 덮고 있던 것이 원인. 처음에는 렌더링 불가로 오진(誤診)하여 조건문만 반복 확인

해결: CSS z-index 조정으로 해결

교훈: 하나의 가설에 집착하지 않고 "렌더링은 되지만 보이지 않을 수도 있다"는 가능성을 열어두는 디버깅 접근법의 중요성

6.7 Spring Validation 어노테이션 미동작 — 의존성 누락

문제: @Valid / @Validated 등 입력값 검증 어노테이션을 컨트롤러에 적용했으나 실제로 동작하지 않고 검증이 통과 되는 현상 발생

원인: Spring Boot 3.x에서는 Bean Validation이 자동으로 포함되지 않으며, spring-boot-starter-validation 의존성을 별도로 추가해야 활성화됨. 어노테이션 자체는 컴파일 오류 없이 선언되지만 의존성이 없으면 실행 시 검증 로직이 동작하지 않음

해결: build.gradle에 spring-boot-starter-validation 의존성 명시적 추가 (PR #325, #326)

교훈: 어노테이션을 추가한 후 실제로 동작하는지 로컬 테스트에서 직접 확인하는 습관이 중요. 코드가 컴파일된다고 해서 의도한 동작이 보장되지 않음

6.8 OS 줄바꿈 문자(CRLF/LF) 차이로 인한 Docker 실행 오류

문제: 로컬(Windows)에서 정상 동작하던 쉘 스크립트가 Docker 컨테이너(Linux) 환경에서 실행 오류 발생

원인: Windows의 줄바꿈 문자는 CRLF(\r\n)이고 Linux는 LF(\n)만 사용함. Git 기본 설정인 `autocrlf=true` 환경에서 Windows에서 작성된 `.sh` 파일이 CRLF로 저장되어 Linux 컨테이너에서 실행 시 \r 문자가 명령어에 포함되어 파싱 오류 발생

해결: `.gitattributes`에 `*.sh text eol=lf` 설정을 추가하여 쉘 스크립트 파일의 줄바꿈을 LF로 강제 통일

교훈: 윈도우-리눅스 혼용 개발 환경에서는 `.gitattributes`로 줄바꿈 문자를 명시적으로 관리해야 함. 로컬에서 정상 동작해도 컨테이너 환경에서 동작하지 않을 수 있음

6.9 Kakao Local API 빈 배열 캐싱으로 인한 TTL 동안 빈 결과 반환

문제: 주변 장소 조회 API에서 특정 카테고리의 검색 결과가 없을 때 빈 배열(`[]`)이 Redis에 캐싱되어, 이후 TTL(1시간)이 만료될 때까지 계속 빈 결과를 반환하는 문제 발생

원인: `fetchWithCache()` 로직에서 Kakao Local API 응답 결과를 성공/실패 여부만 확인하고 캐싱했기 때문에, 정상 응답이지만 결과가 없는 경우(`documents`가 빈 리스트)에도 그대로 Redis에 저장됨. 이후 동일 요청은 캐시 히트로 처리되어 실제 API를 호출하지 않고 빈 배열을 그대로 반환

해결 (PR #283): 캐시 저장 전 결과가 비어있는지 확인하는 조건 추가

```
if (!result.isEmpty()) {
    stringRedisTemplate.opsForValue().set(cacheKey,
    objectMapper.writeValueAsString(result), CACHE_TTL);
}
```

빈 결과는 캐싱하지 않아 다음 요청 시 Kakao API를 재호출하여 최신 결과를 반환하도록 수정

교훈: 캐싱 대상의 유효성 검증은 "API 호출 성공 여부"가 아닌 "반환 데이터의 의미 있는 값 여부"로 판단해야 함. 빈 결과도 정상 응답으로 취급해 캐싱하면 데이터 부재 상황이 TTL 내내 고착됨

6.10 LLM 빈 응답으로 인한 AI 파싱 반복 실패

문제: 운영 로그에 `[OpenAI] 응답 파싱 실패 - Expecting value: line 1 column 1 (char 0)` 에러가 반복 출력되며 AI 분석 결과가 누락됨

원인: `_parse_results()` 메서드가 LLM 응답이 빈 문자열일 때 `json.loads("")`를 호출해 `JSONDecodeError` 발생. 빈 응답이 발생하는 경우는 세 가지였음

- 콘텐츠 필터링으로 인한 응답 잘림
- tool call만 반환되고 최종 텍스트가 없는 경우
- 분석 배치에 대상이 없는 경우

원인 파악이 어려웠던 이유는 실패 시 원본 응답 내용을 로깅하지 않아 재현과 디버깅이 불가능했기 때문

해결 (PR #311):

- 빈 응답 guard 추가 — 빈 문자열이면 예외를 던지지 않고 빈 리스트 반환 후 경고 로그 출력
- 파싱 실패 시 응답 내용 길이와 200자 미리보기를 로그에 포함하여 향후 디버깅 가능하도록 개선

교훈: 에러 로그에 원본 데이터를 함께 남기지 않으면 재현과 원인 파악이 불가능해짐. 파싱 로직 설계 시 예외 케이스(빈 값, null, 예상치 못한 포맷)에 대한 방어 코드와 진단 로그를 함께 작성해야 함

7. KPT 회고

프로젝트 전 기간에 걸쳐 4차례의 스프린트 회고를 진행하며 팀 문화와 개발 프로세스를 지속적으로 개선했습니다.

스프린트 1 — 설계 및 기반 구축

달성도: 8 / 9

구분	내용
Keep	기획~실행 의사결정 주기가 짧아 높은 생산성 유지 · AI 에이전트를 통한 2차 검증으로 안정적 개발 · 팀원 간 장벽 없는 소통으로 잠재적 문제 조기 발견
Problem	컨디션 관리 실패로 몸살 팀원 발생 · 구현 속도 대비 설계에 투자한 시간 부족 · 구현 방향 동기화 부족으로 통합 시 오차 발생
Try	작업 시작 전 각자의 이해와 구현 방향 구두/메신저로 사전 공유 · 코드 착수 전 설계안 공유 및 피드백 (GitHub Discussion 활용) · 트러블슈팅 과정 수시 공유로 문제 반복 방지

팀원 성찰 요약

- **김동현:** MSA 설계 시 기술적 트레이드오프 고려의 중요성 인식. 최고의 기술이 최선의 선택은 아님을 체득
- **이수진:** ADR 작성으로 기술 선택의 근거와 트레이드오프를 문서화하는 과정의 중요성 깨달음. 로컬과 실제 모바일 환경 간 레이아웃 차이 경험
- **권혁태:** 유동적 설계 변경이 효율적임을 학습. CI/CD 파이프라인 및 테스트 환경의 조기 구축이 안정적 개발의 기반임을 확인

스프린트 2 — 핵심 기능 개발

달성도: 24 / 26

구분	내용
Keep	문제 발생 시 즉각 팀장 보고로 해결 시간 단축 · GitHub Projects 칸반 보드로 진행 상황 시각화 · 기술 블로그 작성으로 팀 지식 공유 문화 정착
Problem	GitHub Revert 남용으로 dev-main 브랜치 싱크 붕괴 · 팀원 숙련도 차이를 고려하지 않은 우선순위 배분 · 스테이징 환경 없이 main 직접 배포로 실서비스 장애 직결
Try	main-dev Branch Protection Rule 적용 (1인 이상 승인 필수) · 스테이징 서버 도입으로 배포 전 최종 검증 단계 추가 · 팀원 숙련도 파악 후 작업 방식 설명 선행

팀원 성찰 요약

- **김동현:** 기술 블로그가 개인 학습 기록을 넘어 팀 지식 격차를 좁히는 협업 수단임을 인식

- **이수진:** 커스텀 혹은 분리 기준 확립 — 재사용뿐 아니라 로직 복잡도가 증가하거나 컴포넌트가 여러 역할을 동시에 수행할 때가 분리 시점
- **권혁태:** GitHub Revert의 내부 동작(PR 자동 생성 및 Merge) 이해. Protected 브랜치에서의 올바른 충돌 해소 워크플로우 습득

스프린트 3 — 고도화 및 안정화

달성도: 13 / 13

구분	내용
Keep	즉각적 이슈 공유 문화 지속 · GitHub Projects 칸반 보드 활용 · main 머지 시 코드 리뷰 + 보충 설명 문화 정착
Problem	테스트 코드 없이 로컬 테스트만으로 배포 → 실환경 동작 검증 어려움 · AI 기능 정확도·신뢰도 측정 방안 부재 · MSA 패턴(API Composition 등) 추가 학습 필요
Try	주요 기능 테스트 코드 작성으로 배포 전 안정성 검증 프로세스 마련 · AI 기능 정확도·신뢰도 평가 기준 및 방법론 팀 내 정의 · main·dev 리뷰 승인 프로세스 지속 유지

팀원 성찰 요약

- **김동현:** OpenAI Function Calling + MCP 연동으로 AI 외부 도구 호출 구조 구현. DLQ 재처리 시 토큰 버킷 기반 소프트 한도 판정 방법 체득
- **이수진:** React StrictMode에서 비동기 요청과 Effect 생명주기의 관계 심층 이해. 다양한 디버깅 도구 교차 활용의 중요성 확인
- **권혁태:** **Flux.flatMap**을 활용한 병렬 API 호출, 캐싱 전략(Look Aside / Write Back / Write Through) 차이, API Composition 패턴 학습

스프린트 4 — 마무리 및 운영

달성도: 8 / 11

구분	내용
Keep	Grafana 모니터링 주기적 확인으로 에러·워닝 조기 발견 · 작업 부하 유연한 팀원 간 재배분 · 에러 및 모호한 부분 발생 시 팀원 간 즉각 질문·답변
Problem	모니터링 확인 주기 길고 Slack 연동 알림 없어 실시간 대응 어려움 · AI 파싱 문제 재발 · 경로 서비스 세부 기획 미사전 논의로 팀원 간 구현 방향 불일치
Try	Grafana-Slack 연동으로 에러/워닝 실시간 알림 설정 · 디스크·메모리 인프라 메트릭에 임계치 알림 추가 · 방향이 갈릴 수 있는 기능은 정기 회의 전 미리 의견 정리

팀원 성찰 요약

- **김동현:** 로그 보존 정책 부재가 디스크 포화 → CD 파이프라인 장애로 연쇄되는 과정 직접 경험. 운영 관점을 기능 개발 단계부터 함께 고려해야 함을 체득
- **이수진:** 대체 장소 마커 클릭 시 혼잡도 미표시 버그 해결 — navigation state 제거 후 상세 페이지가 직접 API 호출하도록 수정. 데이터 흐름 설계의 선행 필요성 인식

- 권혁태: @JsonIgnoreProperties, Redis Fallback 로직, ODsay 응답 데이터 가공 등 실전 운영 노하우 습득. 코드 리뷰 도구 활용으로 안정적·예외 고려 코드 작성의 중요성 체감

향후 개선 방향

프로젝트는 당초 6월 중순 마감에서 5월 말로 기한이 단축되어 스프린트 5 일부 항목이 진행 중입니다.

스프린트 5 예정 작업

분류	내용
프론트엔드	모바일 반응형 UI 전면 개선, 혼잡도 통계 차트 구현, 상세 페이지 AI 리포트 연동 완성
테스트	주요 API 단위·통합 테스트 코드 작성, 가상 사용자 시나리오 기반 E2E 테스트
성능 검증	부하 테스트로 실사용 환경에서의 처리량 및 응답 시간 검증

중장기 개선 방향

항목	내용
Circuit Breaker	Resilience4j 적용으로 외부 API 타임아웃 시 Fallback 처리 및 장애 전파 차단
AI 정확도 측정	AI 혼잡 원인 분석 결과에 대한 정확도·신뢰도 평가 기준 및 방법론 수립
Grafana-Slack 알림	에러·워닝 및 인프라 임계치 초과 시 Slack 실시간 알림 자동화
스테이징 환경	운영 서버와 동일한 스테이징 환경 구축으로 배포 전 최종 검증 단계 추가
MCP 기반 AI 고도화	외부 데이터 동적 연동으로 혼잡 원인 분석 정확도 향상

마치며

단번에 프로젝트는 서울시 공공데이터를 기반으로 MSA + EDA 아키텍처를 직접 설계하고 운영까지 경험한 팀 프로젝트입니다. 4차례의 스프린트를 거치며 기술적 역량뿐만 아니라 팀 커뮤니케이션, Git 전략, 인프라 운영, 코드 품질 관리 까지 폭넓은 실무 경험을 쌓을 수 있었습니다. 설계·배포·운영 각 단계에서 맞닥뜨린 문제들을 KPT 회고를 통해 팀 전체의 학습으로 전환한 것이 실제로 많은 도움이 되었습니다.